

The Endowment Guide

How to give a dividend-paying coin a permanent holder that never sells, funded by its own largest holders.

Source: <https://github.com/The-Endowment> (Apache-2.0)

1. The idea in brief

Most meme coins pay holders nothing, so a large holder only gets paid by selling. Coins that pay dividends change that. A large position becomes an income-producing asset, and holding becomes the profitable move. We call holders who accumulate for the income **landlords**: traders create volume, volume pays rent, and rent keeps supply off the market.

An **endowment** takes the idea one step further. Landlords lend it their dividend income, and it turns that income into more of the coin, held in a vault that can never sell. It is a landlord whose commitment is written into code.

2. How an endowment works

1. **Delegate.** A landlord signs one transaction that delegates their dividend token account (for example their PUMP account) to the endowment. Only that one account is involved. Their coin, their SOL and every other token stay untouched.
 2. **Sweep.** When dividends land in the landlord's wallet, anyone can trigger a sweep. It moves the new dividend tokens, above the balance the landlord held when they joined, into the endowment's dividend vault.
 3. **Buy.** The endowment buys the coin in small amounts through the coin's Raydium CPMM pool and locks it in its coin vault. No instruction can ever move coin out of that vault.
 4. **Leave.** A landlord can revoke the delegation at any time, from any wallet, without the endowment's permission.
-

3. The rules

Commitment threshold. Sweeps run only while landlords together hold enough of the coin's supply, 30% by default. Once a day, anyone can run the commitment count. It starts a round, reads every landlord in batches, and closes the round once all are read (or after four hours). Each landlord counts

for the lower of their balance now and at their previous reading, so coin must be held from one count to the next to count (a landlord's first count only records its balance), and only landlords whose delegation is still in place are counted. Between counts, the endowment's **refresher** (a key set in its settings, normally the keeper) checks every landlord, several times a day at times nobody else chooses, sending each pass all at once. Each check can only lower a recorded balance, and drops landlords that have revoked. A landlord counts once it has had three checks since its last count, at least 30 minutes apart; one still waiting for checks when the count reaches it is simply counted later in the round. So coin moved between landlord wallets counts once, unless it lands ahead of every one of those checks in each wallet. With no refresher, nothing counts, and a count can only start after the refresher has checked someone. Every landlord's record, and every check, is on-chain. Sweeps switch on at the threshold and pause if commitment falls below a lower line (25% by default), so small changes don't flip them back and forth.

The refresher is trusted. It can't move anything, but it chooses when to check, so a refresher working with a landlord could time its checks to count the same coin in more than one wallet, or skip landlords. That's why every check is public, each endowment's refresher is shown on the projects page, and the refresher can resign at any time (even after the admin role is renounced). Resigning switches sweeps off at once and voids every check it made, including in a pending settings change. Name a refresher you and your landlords trust, ideally a dedicated keeper key.

Landlords. There is no limit on landlords. Each must hold a minimum stake (0.1% of supply by default); anyone can remove a landlord who no longer holds it or has revoked.

Commitment is per wallet. Everything in a landlord's wallet is committed: all its coin counts toward the threshold, and all new dividend tokens arriving in it are swept. To commit part of a holding, keep the rest in another wallet.

Buying. Each buy is sized by the contract: no larger than the per-buy cap, the daily allowance, or what the pool can absorb within the price-impact limit. Buys are priced against 30 minutes of the pool's own recorded price history (on a very busy pool, all of Raydium's recorded history, never less than 15 minutes), where no single stretch counts for more than a quarter, and wait while the coin is pricier than that average by more than the endowment's price band (5% by default, settable between 1% and 10%). A dip below the average is a good price, so buys go ahead. Every fill must also clear a floor set from the average, so however the price got there, a buy never fills worse than that floor. The contract works out how precisely it has read Raydium's history and includes that in the band and the floor. Tokens sent straight into the pool don't move the average. Buys are spaced by a minimum interval, and a small tip (0.25% by default) pays whoever triggers each buy, which covers the automation's network fees. Sweeps and donations keep a dividend vault to at most three days of what its buys can spend (given the daily allowance, per-buy cap and buy interval); the rest waits in landlords' own accounts, or the donor's vault, until it's needed.

Milestone. Each endowment has a milestone, for \$PENIS 200 million coins bought. After it, contributions keep flowing, and a set share of every buy becomes permanent liquidity in the pool. The liquidity tokens are held by the endowment and can never be withdrawn.

Pause. A guardian key can pause the endowment for up to seven days. A pause can't move funds, lifts on its own, and can't be renewed back to back. Landlords can always leave while paused.

Timelock. Every limit and threshold is bounded by hard-coded limits, and any change, like retiring the endowment, waits 72 hours on-chain before it takes effect. For its first day after that only the admin can apply it; then anyone can, until it expires a week later. Rotating the guardian (which can only pause) is immediate.

Key burn. After a public testing period the admin role is renounced and the program's upgrade key is destroyed. From then on, no one can change the rules or the code. The refresher keeps checking the count, and can only ever resign.

4. Creating an endowment on the shared contract

The shared contract hosts many endowments. Each one has its own vaults, landlords, settings and count, keyed by its coin and its creator, and no endowment can touch another's accounts.

Requirements

- A Raydium CPMM pool pairing your coin with the asset it pays as dividends.
- Dividends pushed to holders' wallets automatically (no claim step). stonk.fun reward coins work this way.
- Either token standard (SPL Token or Token-2022) for the coin and the dividend asset.
- A coin nobody can mint, freeze or take back: no mint or freeze authority, no permanent delegate, and no pausable, non-transferable, confidential or active transfer-hook extension. The dividend asset can't carry those extensions either (a freeze authority is fine). Transfer fees up to 5% and metadata are fine. The contract checks this at creation, including that neither fee is already above the cap.

Steps

1. Choose your settings: buy caps, daily allowance, price-impact limit, price band, minimum buy, buy interval, tip, activation threshold, minimum stake, milestone, and the post-milestone liquidity share.
2. Choose an optional donation to the \$PENIS Endowment: 0%, 0.1%, 0.2% or 0.3% of each buy. It is available for coins paid in PUMP and is locked at creation.

3. Send `create_endowment` from your creator wallet. Put the admin and guardian roles in multisigs you control, such as Squads vaults, and name your keeper's wallet as the refresher (it defaults to the creator).
4. Share your endowment's opt-in link with your landlords, and run a keeper (section 6), or rely on anyone who wants the tip.

Creation opens on the shared contract once its upgrade key has been destroyed, so every project runs on code that can never change.

5. Deploying your own copy

Everything is Apache-2.0, so you can run your own deployment:

1. Clone the `endowment` repository. Install Rust, the Solana CLI and Anchor 1.1.2.
 2. `./scripts/test.sh` builds the program and runs the unit and integration tests. The integration tests use snapshots of a real Raydium pool.
 3. Update the program ID, and set `FLAGSHIP_CREATOR` (and `FLAGSHIP_COIN_MINT`) if you want donations to go to your own flagship; while the creator is the placeholder, donations are disabled. Deploy with a fresh upgrade authority held in a multisig.
 4. Clone the `website` repository for the opt-in page and keeper, and point it at your program ID and instance.
 5. Run your own review before launch, test with small caps, then renounce the admin role and destroy the upgrade key.
-

6. Running the keeper

The keeper is a small service that triggers the endowment's permissionless instructions. It holds no special power, and anyone can run one.

- **Sweeps.** Run after every dividend drop (for example from a webhook on the dividend distributor), with a timer as a fallback. Each landlord is swept on its own, so one failure never blocks the others.
- **Buys.** Try every 5 minutes. Once the minimum interval has passed and the vault holds at least the minimum buy, and the daily allowance has refilled to at least the minimum buy, simulate `buyback`; if it would succeed, send it with `min_out` at the simulated fill less about 1%. The contract does the sizing and pricing, and prices sometimes run above the band for a while, so frequent attempts catch the moments when they're inside it. A donating endowment's buy passes the flagship's coin mint and config as its last two accounts.

- **Refresh.** As the endowment's refresher, several times a day at random times, run `refresh_landlords` over every landlord in a fresh random order, sending every batch before confirming any. A landlord counts after three of these checks, 30 minutes apart, since its last count.
- **Count.** Once a day, run a refresh pass and `begin_count`, then `count_landlords` in batches of about eight landlords, then `finish_count`. Landlords still waiting for checks stay in the round: check them again (30 minutes after their last check) and count them until all are counted or the round times out. A batch fits a normal transaction, so no lookup table is needed.

The website repository includes a ready-made keeper with setup notes in `docs/keeper.md`.

7. Security notes

- **What landlords approve.** Landlords approve an unlimited delegation on one dividend account. The program only ever moves tokens above the landlord's recorded balance, and only into its own vault. Before the key burn, the upgrade key holder could change the program, so keep that key in a multisig and announce every upgrade.
- **Recommend a dedicated wallet.** Once a landlord joins, all new dividend tokens arriving in that account go to the endowment, whatever their source. A wallet that holds only the coin keeps this simple.
- **Mint controls.** The contract refuses coins with mint, freeze, pause or take-back powers at creation. For what it allows (fee authorities, a dividend freeze authority, a hook authority with no hook set), sweeps and buybacks both stop, rather than lose anything, if a fee rises above the cap, a hook is switched on, a vault is frozen or the pool stops trading. New dividends then stay with landlords, and the vault never holds more than three days of buys. The \$PENIS mint has a transfer-fee authority and the PUMP mint a hook authority, both held by their issuers; the projects page lists these for every endowment.
- **Test with small caps first,** and review the code yourself or with your own agents before you launch.